

Bubble Sort

1 What is Bubble Sort?

Bubble Sort is a simple sorting algorithm that repeatedly compares **adjacent elements** and swaps them if they are in the wrong order.

For sorting in ascending order:

If $A[j] > A[j + 1]$, swap them

After every complete pass, the largest unsorted element moves to the end of the array.

This movement of the largest element towards the end is similar to a bubble rising to the surface. Hence, the name **Bubble Sort**.

2 Basic Idea of Bubble Sort

Consider:

$A = [5, 3, 8, 4, 2]$

We compare adjacent elements:

5 and 3

Since:

$5 > 3$

we swap them:

$[3, 5, 8, 4, 2]$

Next:

5 and 8

Since:

$5 < 8$

no swap is performed.

Next:

8 and 4

Since:

$8 > 4$

swap:

[3, 5, 4, 8, 2]

Next:

8 and 2

Since:

$8 > 2$

swap:

[3, 5, 4, 2, 8]

At the end of the first pass:

[3, 5, 4, 2, 8]

The largest element, 8, has reached the last position.

3 Bubble Sort Algorithm

The basic algorithm for ascending order is:

```
1 BubbleSort(A, n)
2
3 for i = 0 to n-2
4
5     for j = 0 to n-i-2
6
7         if A[j] > A[j+1]
8
9             swap(A[j], A[j+1])
```

The corresponding C++ program is:

```
1 void bubbleSort(int A[], int n)
2 {
3     for(int i = 0; i < n - 1; i++)
4     {
5         for(int j = 0; j < n - i - 1; j++)
6         {
7             if(A[j] > A[j + 1])
8             {
9                 int temp = A[j];
10                A[j] = A[j + 1];
11                A[j + 1] = temp;
12            }
13        }
14    }
15 }
```

4 Understanding the Two Loops

Bubble Sort contains two loops.

4.1 Outer Loop

The outer loop controls the number of passes:

```
1 for(int i = 0; i < n - 1; i++)
```

Therefore:

$$i = 0, 1, 2, \dots, n - 2$$

The number of iterations is:

$$(n - 2) - 0 + 1$$

Using the formula:

$$u - l + 1$$

we get:

$$(n - 2) - 0 + 1 = n - 1$$

Therefore, the outer loop executes:

$$n - 1 \text{ times}$$

Hence, Bubble Sort can have:

$$n - 1 \text{ passes}$$

5 Number of Comparisons in Each Pass

The inner loop is:

```
1 for(int j = 0; j < n-i-1; j++)
```

For each value of i , the upper limit changes.

The comparison is:

$$A[j] > A[j + 1]$$

This is the **basic operation** of Bubble Sort.

5.1 Pass 1

For:

$$i = 0$$

the loop is:

```
1 for(int j = 0; j < n-1; j++)
```

Therefore:

$$j = 0, 1, 2, \dots, n - 2$$

Using:

$$u - l + 1$$

we get:

$$(n - 2) - 0 + 1$$

$$= n - 1$$

Therefore, Pass 1 makes:

$$n - 1 \text{ comparisons}$$

5.2 Pass 2

For:

$$i = 1$$

the loop is:

```
1 for(int j = 0; j < n-2; j++)
```

Therefore:

$$j = 0, 1, \dots, n - 3$$

Number of comparisons:

$$(n - 3) - 0 + 1$$

$$= n - 2$$

Therefore:

$$n - 2 \text{ comparisons}$$

5.3 Pass 3

For:

$$i = 2$$

number of comparisons:

$$(n - 4) - 0 + 1$$

$$= n - 3$$

Therefore:

$$\boxed{n - 3 \text{ comparisons}}$$

5.4 General Pattern

The number of comparisons in each pass is:

Pass	Comparisons
1	$n - 1$
2	$n - 2$
3	$n - 3$
$\vdots$	$\vdots$
$n - 1$	1

Therefore, the total number of comparisons is:

$$(n - 1) + (n - 2) + (n - 3) + \cdots + 1$$

6 Total Number of Comparisons

Using summation notation:

$$C(n) = \sum_{i=0}^{n-2} (n - i - 1)$$

Let us calculate this using the **upper limit – lower limit + 1** idea.

For a fixed value of i , the inner loop is:

$$j = 0$$

to:

$$j = n - i - 2$$

Therefore, the number of comparisons is:

$$(n - i - 2) - 0 + 1$$

$$= n - i - 1$$

Hence:

$$C(n) = \sum_{i=0}^{n-2} (n - i - 1)$$

The terms are:

$$(n - 1) + (n - 2) + \cdots + 2 + 1$$

This is:

$$1 + 2 + 3 + \cdots + (n - 1)$$

Using:

$$\sum_{k=1}^m k = \frac{m(m+1)}{2}$$

where:

$$m = n - 1$$

we get:

$$C(n) = \frac{(n-1)n}{2}$$

Therefore:

$$\boxed{C(n) = \frac{n(n-1)}{2}}$$

Thus, the total number of comparisons is:

$$\boxed{\frac{n(n-1)}{2}}$$

7 Time Complexity

Hence, the basic Bubble Sort algorithm has:

$$\boxed{\text{Worst-case Time Complexity} = O(n^2)}$$

8 Example: Trace Bubble Sort

Consider the following array:

$$A = [5, 3, 8, 4, 2]$$

Here:

$$n = 5$$

The number of passes is:

$$n - 1 = 5 - 1 = 4$$

The expected number of comparisons is:

$$\begin{aligned} & \frac{n(n-1)}{2} \\ &= \frac{5(5-1)}{2} \\ &= \frac{5 \times 4}{2} \\ &= 10 \end{aligned}$$

Therefore:

10 total comparisons

Now let us trace the algorithm.

8.1 Pass 1

Initial array:

$$[5, 3, 8, 4, 2]$$

There are:

$$5 - 1 = 4$$

comparisons.

Comparison 1

Compare:

$$5 \text{ and } 3$$

Since:

$$5 > 3$$

swap:

[3, 5, 8, 4, 2]

Comparison 2

Compare:

5 and 8

Since:

$5 < 8$

no swap:

[3, 5, 8, 4, 2]

Comparison 3

Compare:

8 and 4

Since:

$8 > 4$

swap:

[3, 5, 4, 8, 2]

Comparison 4

Compare:

8 and 2

Since:

$8 > 2$

swap:

[3, 5, 4, 2, 8]

Therefore, after Pass 1:

[3, 5, 4, 2, 8]

The largest element 8 is now in its correct position.

Total comparisons so far:

4

8.2 Pass 2

Now 8 is already sorted, so we do not compare it again.

The array is:

[3, 5, 4, 2, 8]

Number of comparisons:

$$n - 2 = 3$$

Comparison 5

Compare:

3 and 5

No swap:

[3, 5, 4, 2, 8]

Comparison 6

Compare:

5 and 4

Swap:

[3, 4, 5, 2, 8]

Comparison 7

Compare:

5 and 2

Swap:

[3, 4, 2, 5, 8]

Therefore, after Pass 2:

[3, 4, 2, 5, 8]

The next largest unsorted element, 5, is now in its correct position.

Total comparisons so far:

7

8.3 Pass 3

Array:

[3, 4, 2, 5, 8]

Number of comparisons:

$$n - 3 = 2$$

Comparison 8

Compare:

3 and 4

No swap:

[3, 4, 2, 5, 8]

Comparison 9

Compare:

4 and 2

Swap:

[3, 2, 4, 5, 8]

Therefore, after Pass 3:

[3, 2, 4, 5, 8]

Total comparisons so far:

9

8.4 Pass 4

Array:

[3, 2, 4, 5, 8]

Only one comparison is required:

$$n - 4 = 1$$

Compare:

3 and 2

Since:

$$3 > 2$$

swap:

$[2, 3, 4, 5, 8]$

Therefore, after Pass 4:

$[2, 3, 4, 5, 8]$

Total comparisons:

$$9 + 1 = 10$$

Therefore:

10 comparisons

9 Complete Trace

Pass	Array after Pass	Comparisons in Pass	Total Comparisons
Initial	$[5, 3, 8, 4, 2]$	—	0
1	$[3, 5, 4, 2, 8]$	4	4
2	$[3, 4, 2, 5, 8]$	3	7
3	$[3, 2, 4, 5, 8]$	2	9
4	$[2, 3, 4, 5, 8]$	1	10

Therefore:

Sorted Array = $[2, 3, 4, 5, 8]$

and:

Total Comparisons = 10

10 General Formula for Number of Passes

Number of Passes = $n - 1$

11 General Formula for Comparisons

The number of comparisons in each pass is:

$$n - 1, n - 2, n - 3, \dots, 2, 1$$

Therefore:

$$\text{Total Comparisons} = (n - 1) + (n - 2) + \dots + 2 + 1$$

Using the summation formula:

$$\sum_{k=1}^{n-1} k = \frac{(n-1)n}{2}$$

we obtain:

$$\text{Total Comparisons} = \frac{n(n-1)}{2}$$

12 Summary

For the basic Bubble Sort algorithm:

$$\text{Number of Passes} = n - 1$$

Comparisons in each pass are:

$$n - 1, n - 2, n - 3, \dots, 1$$

Total comparisons:

$$\frac{n(n-1)}{2}$$

Therefore:

$$O(n^2)$$

13 Quick Revision Table

Concept	Result
$A[j] > A[j + 1]$	Basic operation
Number of passes	$n - 1$
$n - 1$	Comparisons in Pass 1
Comparisons in Pass 2	$n - 2$
$n - 3$	Comparisons in Pass 3
Comparisons in final pass	1
$\frac{n(n-1)}{2}$	Total comparisons
Worst-case time complexity	$O(n^2)$

Efficiency Analysis

Time complexity

$$= \sum_{i=0}^{n-1} \left(\sum_{j=0}^{n-i-1} 1 \right)$$

$$= \sum_{i=0}^{n-1} (n-i-1+1)$$

$$= \sum_{i=0}^{n-1} (n-i)$$

$$= (n-0+1)(n-i)$$

$$= (n+1)(n-i)$$

$$= n^2 - ni$$

$$\underline{\underline{= n^2}}$$

$$\Rightarrow \underline{\underline{O(n^2)}}$$